

```
subfolders. Returns a list of files that the match function
returned True for."""
matchedFiles = [] # This list holds all the matches.
folder = os.path.abspath(folder) # Use the folder's absolute path.
```

The `walk()` function has two parameters: `folder` is a string of the base folder to search (we can pass `'.'` to refer to the current folder the Python program is run from), and `matchFunc` is a Python function that is passed a filename and returns `True` if the function says it is a search match. Otherwise, the function returns `False`.

The next part of the function examines the contents of `folder`:

Python

```
# Loop over every file and subfolder in the folder:
for name in os.listdir(folder):
    filepath = os.path.join(folder, name)
    if os.path.isfile(filepath):
```

The `for` loop calls `os.listdir()` to return a list of the contents of the `folder` folder. This list includes all files and subfolders. For each file, we create the full, absolute path by joining the folder with the name of the file or folder. If the name refers to a file, the `os.path.isfile()` function call returns `True`, and we'll check to see if the file is a search match:

Python

```
# Call the match function for each file:
if matchFunc(filepath):
    matchedFiles.append(filepath)
```

We call the match function, passing it the full absolute filepath of the `for` loop's current file. Note that `matchFunc` is the name of one of `walk()`'s parameters. If `hasEvenByteSize()`, `hasEveryVowel()`, or another function is passed as the argument for the `matchFunc` parameter, then that is the function `walk()` calls. If `filepath` contains a file that is a match according to the matching algorithm, it's added to the `matches` list:

Python